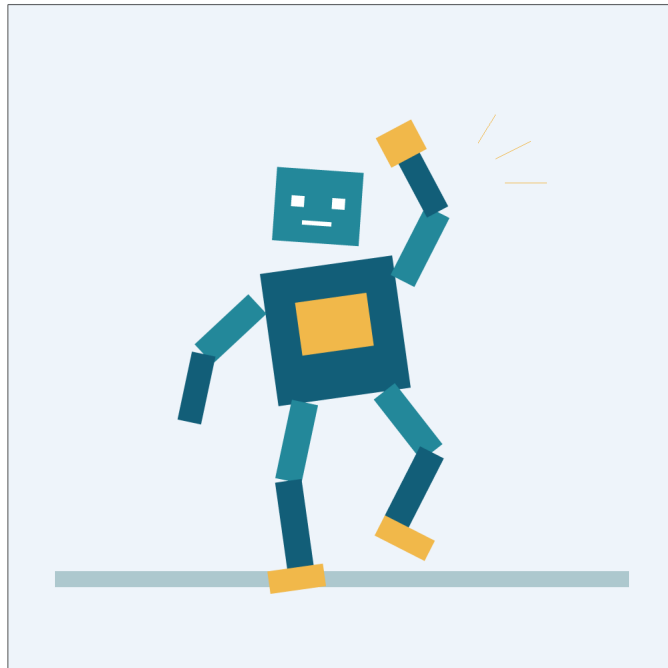


Rasterizer

Team nineteen

<https://cal-cs184-student.github.io/hw1-nineteen-writeup/>

A software pipeline for converting simplified SVG geometry into pixels. The implementation covers solid triangles, grid supersampling, hierarchical 2D transforms, barycentric color interpolation, and texture mapping with nearest, bilinear, and mipmap filtering.



A waving, stepping cubeman built from nested translations, rotations, and scales. Details in Task 3.

Overview

The renderer first transforms SVG geometry into screen coordinates. Each triangle then tests sample locations within its clipped bounding box and writes covered colors into a supersample buffer. Color triangles interpolate vertex colors; textured triangles interpolate UV coordinates and select texture samples. Finally, the renderer averages the samples of each pixel into an 8-bit RGB framebuffer.

The comparisons illustrate two distinct sources of aliasing: insufficient coverage samples at geometric edges, and insufficient filtering of texture detail. More screen samples improve edge coverage; bilinear interpolation improves texture magnification; mipmaps suppress detail that is too fine for the projected image. These controls can be combined, but solve different parts of the sampling problem.

All render images in this report were saved with the renderer's S hotkey as lossless 1600×1200 PNGs. The enlarged inspector panels repeat the top-right area of those same PNGs. No operating-system screenshots are used as assignment figures.

Code is kept in the private homework repository. This public report contains only the write-up, rendered results, and supporting SVG/image assets. Optional extra credit is not claimed.

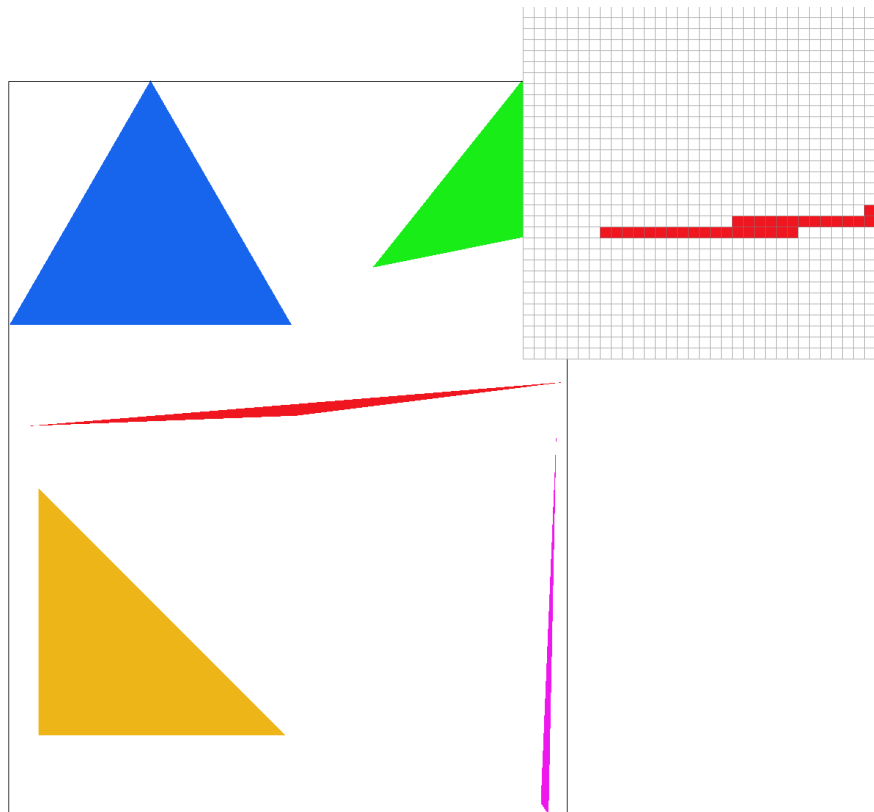
Drawing single-color triangles

For a triangle, the minimum and maximum vertex coordinates define an axis-aligned bounding box. The rasterizer rounds those limits outward and clips them to the framebuffer, so an offscreen triangle cannot write outside the buffer. At one sample per pixel, the location tested is $(x + 0.5, y + 0.5)$.

Three signed edge functions determine coverage. Each measures which side of a directed edge contains the sample. If all three values are nonnegative, or all are nonpositive, the sample is inside. Accepting zero includes boundary samples; accepting either common sign supports clockwise and counterclockwise vertices. A zero-area triangle is skipped.

$$E(A,B,P) = (B_x - A_x)(P_y - A_y) - (B_y - A_y)(P_x - A_x)$$

Only samples inside the clipped bounding box are visited. Each one requires three edge tests and a constant amount of work, giving $O(B)$ work at one sample per pixel, where B is the number of pixels in that box. This meets the bounding-box requirement and avoids scanning unrelated framebuffer pixels. The same coverage helper is reused by all three triangle types.



basic/test4.svg at the default viewpoint, 1 sample per pixel. The inspector is centered on the narrow left tip of the red triangle.

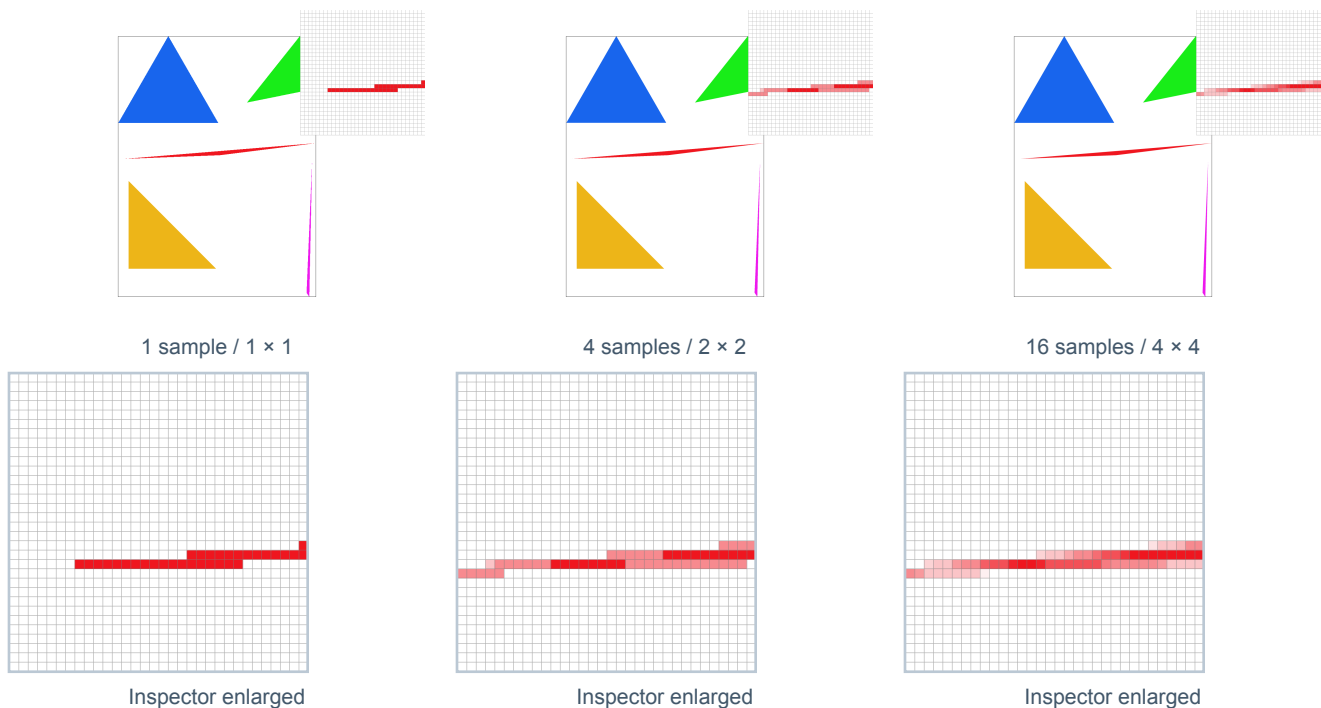
At one sample, each pixel is either covered or uncovered. The red tip therefore breaks into a staircase, and very thin portions can miss all sample centers. Task 2 shows how additional samples soften that transition.

Antialiasing by supersampling

For a sample rate s , the renderer uses an $n \times n$ grid with $n = \sqrt{s}$. The sample centers within a pixel are $(x + (i + 0.5)/n, y + (j + 0.5)/n)$. Every covered sample receives the primitive's color independently, preserving the drawing order even when triangles overlap within one output pixel.

The `sample_buffer` stores $\text{width} \times \text{height} \times s$ Color values. Pixel (x,y) begins at $(y \times \text{width} + x) \times s$; local sample (i,j) has offset $j \times n + i$. Both framebuffer resizing and sample-rate changes reallocate this storage. Clearing makes every sample white. Points and lines fill all samples of their output pixels with the same color.

After every primitive is drawn, `resolve_to_framebuffer` averages the s colors for each pixel, clamps each channel to $[0,1]$, and converts to an 8-bit value. A partly covered pixel blends the triangle with the colors of its uncovered samples, approximating area coverage.



All three images use the default viewpoint and the same inspector position on `basic/test4.svg`. At 1 sample the tip has abrupt solid-red steps. At 4 samples, pale red pixels appear around the edge. At 16 samples there are more intermediate coverage values and the tip is more continuous. Supersampling reduces these artifacts at the cost of more triangle work and sample memory.

A boundary test at rate 9 exposed a numerical issue: separately rounded thirds could put an exact edge sample just outside. Coverage is now evaluated after scaling the geometry by $2n$, so sample centers have odd integer coordinates. Barycentric weights use the correspondingly scaled area. The regression now covers six of nine samples in a half-pixel test triangle, including its diagonal boundary.

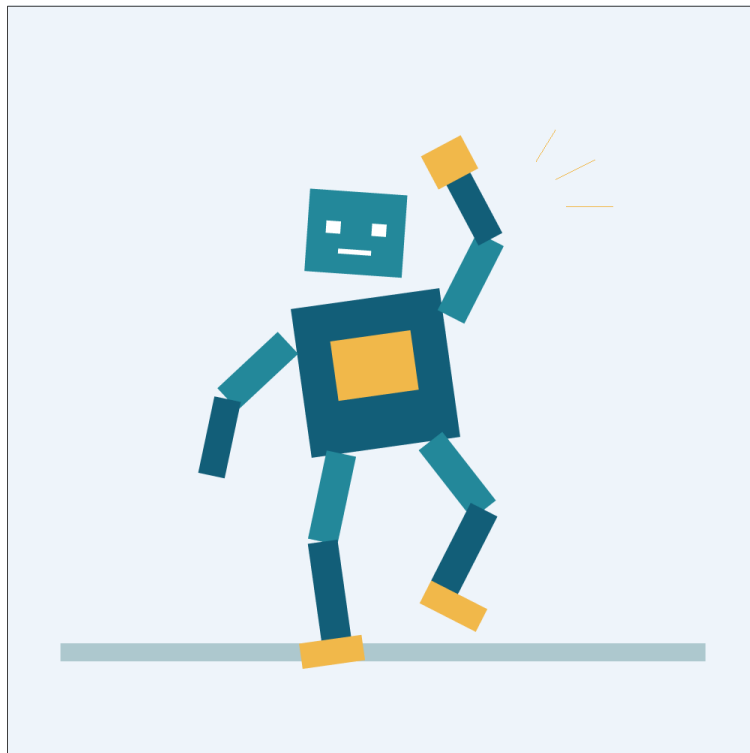
Hierarchical transforms

Translation, scale, and rotation use 3×3 homogeneous matrices. Translation places dx and dy in the last column; scale places sx and sy on the first two diagonal entries. Rotation converts degrees to radians and uses cosine and sine. The final homogeneous coordinate remains 1 for these affine transformations.

$$T = [1 \ 0 \ dx; 0 \ 1 \ dy; 0 \ 0 \ 1] \quad S = [sx \ 0 \ 0; 0 \ sy \ 0; 0 \ 0 \ 1]$$

$$R = [\cos \theta \ -\sin \theta \ 0; \sin \theta \ \cos \theta \ 0; 0 \ 0 \ 1]$$

The custom cubeman waves while taking a step. Its torso is tilted, one arm bends upward at the shoulder and elbow, and one knee bends while the other leg reaches toward the ground. A teal-and-gold palette separates the body, limbs, hands, and feet. Small straight rays emphasize the waving hand.



`my_robot.svg` rendered at 16 samples per pixel. The illustration uses supported rectangles, groups, and straight lines.

The outer translation positions the figure; a parent rotation tilts its entire body. Shoulder and hip translations establish joint origins. Each limb's rotation acts about its local origin, and the elbow or knee group is nested inside its parent segment. Moving the shoulder therefore also moves the forearm and hand. The head and raised hand use local scale groups to change proportions.

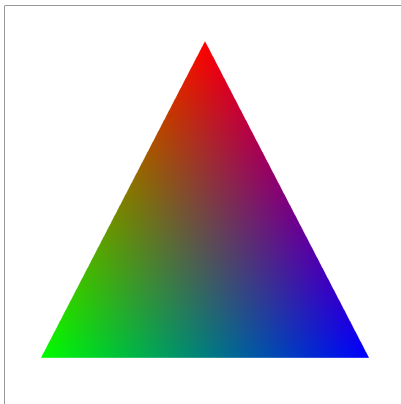
With column-vector multiplication, the rightmost transform acts first. For example, a child point is scaled in its local coordinates, then rotated, then translated into its parent's coordinates. Positive mathematical rotation appears clockwise on the screen because SVG screen y increases downward. The downloadable `my_robot.svg` is included in the report repository.

Barycentric coordinates

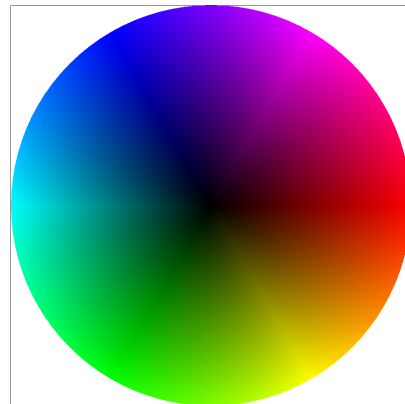
Barycentric coordinates describe a point as weights attached to the triangle's three vertices. The weights sum to one. At a vertex, that vertex's weight is one and the others are zero; on an edge, the opposite vertex's weight is zero. Inside the triangle, all three weights are nonnegative.

$$P = \alpha A + \beta B + \gamma C \quad \alpha + \beta + \gamma = 1$$

Each weight is obtained by dividing the signed area of the opposite subtriangle by the signed area of the whole triangle. The rasterizer reuses its edge functions for these area ratios. It then computes the sample color as $\alpha C_A + \beta C_B + \gamma C_C$. This gives a continuous blend across the triangle, evaluated separately at every covered supersample.



RGB triangle / 1 sample



basic/test7.svg / 1 sample

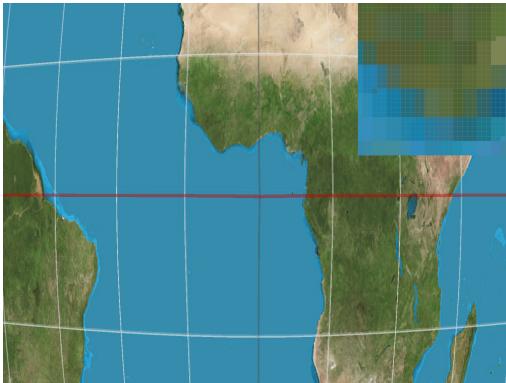
In the custom triangle, the top vertex is red, the lower-left vertex green, and the lower-right vertex blue. Near the top, the red weight dominates. The lower edge contains no red contribution; its colors blend green and blue. At the centroid, the three weights are $1/3$, producing equal RGB components. Along each edge, only its two endpoint colors contribute.

The color wheel on the right is shown at the default viewpoint and sample rate 1, as required. Neighboring colored triangles join into a smooth wheel. The custom `rgb_triangle.svg` is also included so the three vertex weights can be inspected in isolation.

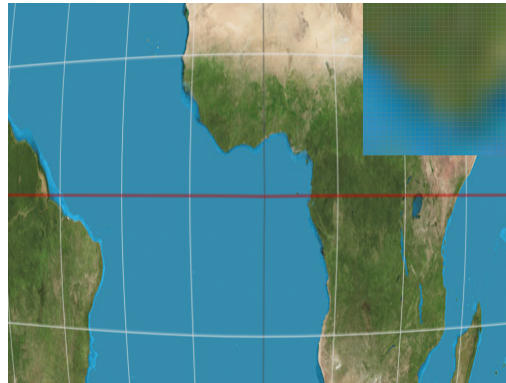
Pixel sampling for texture mapping

Texture mapping assigns each screen sample a position (u,v) in an image. The rasterizer first computes barycentric weights and uses them to interpolate the triangle's vertex UVs. Within a selected mip level, normalized UV coordinates map to texel coordinates $(u \times (\text{width} - 1), v \times (\text{height} - 1))$. Coordinates are clamped to the texture boundary.

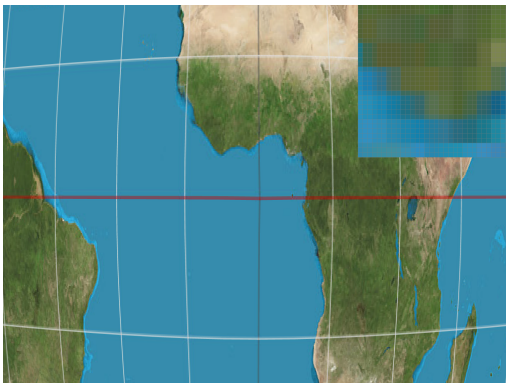
Nearest sampling rounds that position to the closest texel and returns its color. Bilinear sampling finds the four neighboring texels, interpolates horizontally in each row, then interpolates vertically between the two results. At the final row or column, neighbor indices clamp to the edge, so UV values of exactly 1 remain valid.



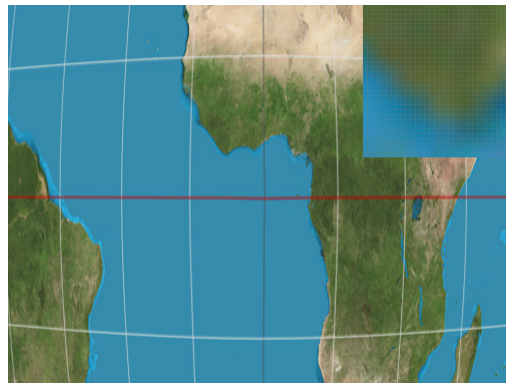
Nearest / 1 sample



Bilinear / 1 sample



Nearest / 16 samples



Bilinear / 16 samples

These captures use `texmap/test1.svg`, `L_ZERO`, the same magnified view, and the same inspector position on the Gulf of Guinea coast. Nearest at 1 sample shows abrupt blocks of repeated texel colors. With 16 samples, averaging smooths some boundaries between those blocks. Bilinear at 1 sample already gives smoother transitions; at 16 samples the remaining changes are more subtle.

The largest nearest-versus-bilinear difference occurs under magnification, when one texel covers several screen pixels, especially across high-contrast texture features. Bilinear blends adjacent texels instead of replicating a single color over each block. Under strong minification, four texels may still cover too little of the pixel footprint; mipmaps address that limitation in Task 6.

Level sampling with mipmaps

A mipmap stores prefiltered versions of a texture at decreasing resolutions. Level sampling chooses a resolution appropriate to the texture area covered by a screen pixel. This reduces high-frequency detail before it can alias into misleading low-frequency patterns.

For each covered sample, the renderer constructs UV values at (x,y) , $(x+1,y)$, and $(x,y+1)$. Because barycentric interpolation is affine across the triangle, the one-pixel UV changes can be computed once per triangle and added to each sample's UV. These offsets stay one full screen pixel apart even when supersampling is enabled.

$$dx = ((ux - u)W, (vx - v)H) \quad dy = ((uy - u)W, (vy - v)H)$$

$$\rho = \max(\text{length}(dx), \text{length}(dy)) \quad \text{level} = \log_2(\max(1, \rho))$$

W and H are the full-resolution texture dimensions. The level is clamped to the available mip range. L_ZERO always uses level 0. L_NEAREST rounds the continuous level. L_LINEAR samples $\text{floor}(\text{level})$ and the next available level, then blends them by the fractional part. Pixel sampling is selected independently at each level: bilinear pixel sampling combined with linear level sampling is trilinear filtering.

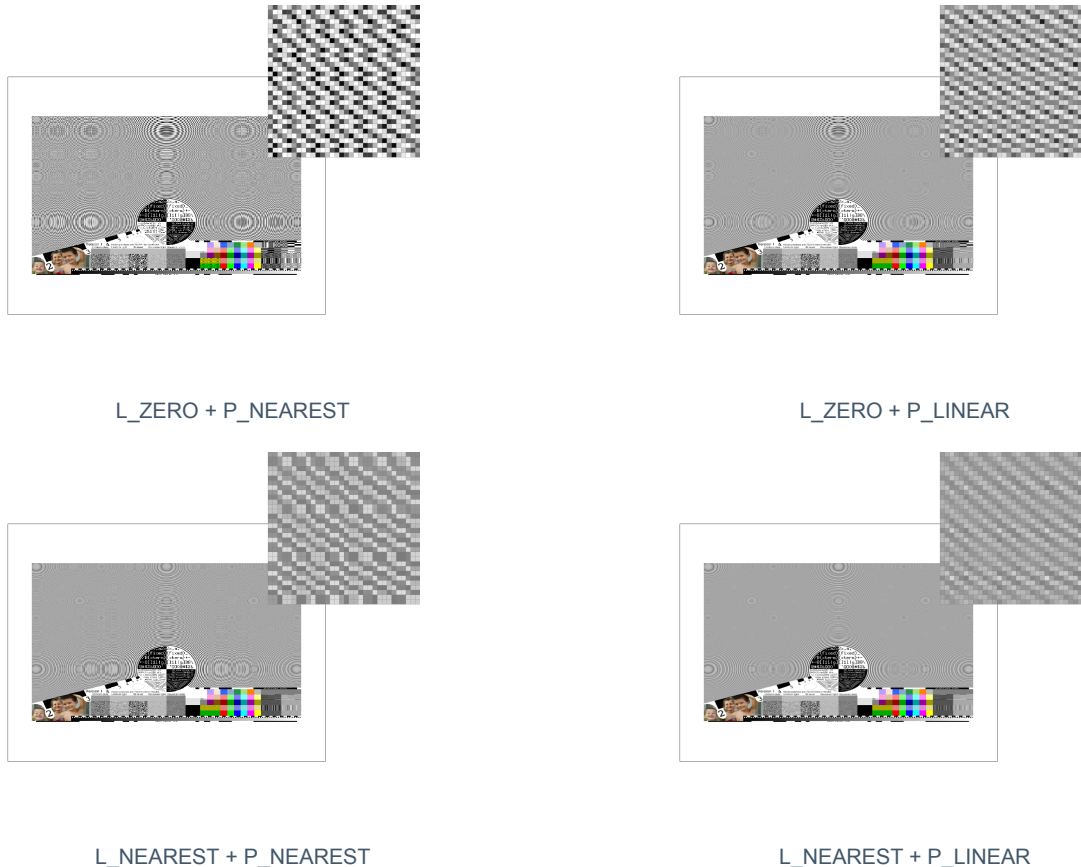
Control	Work / memory	Antialiasing effect
More screen samples	Rasterization, shading, and sample storage grow with s. The buffer uses 12WHs bytes for float RGB.	Improves geometry edges and averages texture samples; high cost at large s.
Nearest → bilinear pixels	1 → 4 texel reads per selected level; no extra persistent texture storage.	Smooths magnified texel transitions; limited protection against strong minification.
Mip levels	A square power-of-two chain approaches 4/3 of base texture storage. Linear level blending samples two levels.	Prefilters minified textures; nearest levels can pop, while linear level blending smooths transitions.

The mip storage estimate assumes a square 2D texture: $1 + 1/4 + 1/16 + \dots = 4/3$. Thin or rectangular textures have a different ratio once one dimension reaches one. MipLevel objects are accessed by reference so sampling never copies entire images.

During validation, the starter mip-generation loop was found to skip its final levels, leaving their texels black. The loop now fills every allocated level through the smallest image. A constant-color 4×4 texture must remain the same color at 2×2 and 1×1 ; that regression passes. No anisotropic filtering or other optional extra-credit filter is claimed.

Mipmap comparison

The custom PNG is Zone plate boys by AlbertCahalan, sourced from Wikimedia Commons. Its fine rings, line patterns, and text make undersampling visible. A custom two-triangle SVG maps the complete 1200×900 image onto a rectangle; all four captures hold the viewpoint, inspector, and sample rate 1 fixed.



L_ZERO with nearest sampling produces conspicuous false ring patterns and high-contrast noise in the inspector. Bilinear sampling at level zero reduces the contrast but still aliases. The nearest mip level removes more high-frequency detail; combining it with bilinear pixel sampling gives the smoothest of these four results. This trades fine detail for a more stable representation at the chosen output size. Some aliasing remains because an isotropic mip estimate is only an approximation to the pixel footprint.

The displayed rectangle is 850×500 framebuffer pixels. Its UV footprint is $\max(1200/850, 900/500) = 1.8$ texels per screen pixel, giving level ≈ 0.848 and nearest level 1. Thus the level-zero and nearest-level columns actually sample different resolutions.

Validation: 40 committed behavioral checks pass with address/undefined-behavior sanitizers. Independent review added 1,469 checks for winding, derivatives, and odd/thin mip chains. Of the 32 original reference PNGs, 27 match byte-for-byte. The color wheel differs at five pixels by one channel unit; the two thin-triangle stress scenes differ at a small number of boundary pixels. Lion and sphere references include partially transparent strokes, which the supplied starter and assignment do not require.

Image credit: AlbertCahalan, "Zone plate boys" (2007), Wikimedia Commons, CC BY-SA 3.0. The source PNG is unmodified; the Task 6 renders are scaled/filtered derivatives under the same license. Source, license, and downloadable scene links appear below. Implementation, testing, and report drafting used Codex assistance.

[Texture source](#) · [CC BY-SA 3.0 license](#)